

Eje B — Robustez y consistencia del flujo de optimización

Estado

Implementado y validado el 5 de septiembre de 2026 sobre una AMD Radeon RX 9070 XT con ROCm 7.2 y `torch.compile`.

El eje B no tiene configuración independiente: extiende `AccelConfig` y sólo puede activarse junto al preset del eje D:

```
arcagi/bin/python profile_parallel_train.py \
  --label eje_b_demo30_candidate \
  --accel-preset compile \
  --eje-b \
  --gpu-mode sysfs \
  -- \
  --split training \
  --demo 30 \
  --iterations 2000 \
  --seeds 0,1,2,3 \
  --max-workers 6 \
  --postprocess-stride 4
```

La CLI rechaza `--eje-b` si el preset solicitado no es `compile` o si el modo efectivo no es `compile_mode='default'`.

Implementación

Free-bits por hoja

Para cada una de las 18 hojas del multitensor se suma primero su tensor KL y luego se aplica el umbral:

$$L_{KL,t} = \sum_i \max(KL_i, \tau_t), \quad \tau_t = \tau_0 \left(1 - \frac{t}{T-1}\right).$$

El valor candidato es $\tau_0 = 2$ nats. Es una elección experimental de esta implementación, no un valor prescrito por el paper. El umbral elimina el gradiente compresivo cuando $KL_i < \tau_t$; no obliga por sí solo a que una hoja aumente su KL.

Se registran por separado la KL raw, la KL efectiva, τ_t y el número de hojas bajo el floor. En el demo de 30 tareas:

- el schedule recorrió exactamente $2 \rightarrow 0$;
- hubo al menos una hoja bajo el floor en el 66,38 % de los pasos;
- media de 6,139 hojas bajo el floor;
- máximo observado de 16 hojas;
- slack medio entre KL efectiva y raw: 0,973 nats.

Curriculum de demostraciones

Cada demostración acumula la reconstrucción de su input y output. La dificultad histórica es una EMA detached de NLL por píxel, estandarizada dentro del puzzle. Los pesos son:

$$w_{i,t} = n_{train} \text{softmax}(\beta_t d_i), \quad \beta_t = \beta_{max} \frac{t}{T-1}.$$

Por construcción, la media de los pesos es uno. Las entradas de test conservan peso uno y las salidas de test nunca participan en la pérdida.

En las 120 trayectorias del demo, la desviación absoluta media respecto a peso uno fue 0,4338; el rango global observado fue `[0.1041, 3.7515]`.

Multi-semilla durable

Cada `(task, seed)` es un job independiente del scheduler. Phase 1 mide una sola instancia eager por tarea; Phase 2 asigna a cada seed la cuota normal de esa tarea. Los parciales se guardan atómicamente bajo:

```
<state-dir>/<split>/eje_b/<fingerprint>/<task>/seed_<seed>.json
```

La huella incluye pasos, stride, configuración B y versión del scorer, pero no ajustes ejecutivos del eje D. Así se puede reanudar un ensemble sin mezclar resultados incompatibles y sin repetir seeds terminadas.

Selección pass@2 target-only

No se implementó el selector aprendido cross-puzzle: habría usado ground truth de otras tareas y cambiaría la promesa target-only.

Cada seed conserva el scorer original. Para fusionar:

1. `attempt_1` es la solución con mayor evidencia de consenso acumulada mediante `logaddexp` entre seeds.
2. `attempt_2` es la solución distinta con mayor evidencia normalizada dentro de cualquier seed. Esto reserva una apuesta para una trayectoria fuerte que el consenso de las demás semillas podría diluir.

La política se denomina `consensus_plus_seed_normalized_diversity_v2`. Usa sólo scores internos, nunca ground truth.

Validación

Suite automatizada

```
71 tests OK antes de la fusión v2
25 tests focalizados OK después de la fusión v2
```

La cobertura incluye schedules, gradientes de free-bits, curriculum detached, jobs por seed, fingerprints, resume, colisiones, fusión, metadata y contratos del perfilador.

Smoke compilado y resume

Una tarea, dos seeds y cinco pasos:

- `compile_mode='default'` confirmado;
- dos jobs y diez optimizer steps;
- cuatro contribuciones por iteración en el NPZ;
- dos parciales independientes;
- replay con `--resume`: dos jobs cargados, cero pendientes y Phase 2 de 0,0 s;
- submission reanudada idéntica byte a byte.

Gate de 11 tareas

Métrica	Control D	D+B
Tareas resueltas	11/11	11/11
Seeds	1	4
Pasos	22.000	88.000
Resultado por seed	—	9/11, 9/11, 11/11, 9/11

El gate estricto se superó. La mejor seed resolvió todas las tareas y la fusión mantuvo 11/11.

Demo de 30 tareas

Métrica	Control D	D+B raw	D+B fusión v2 materializada
pass@2	11/30 (36,7 %)	11/30 (36,7 %)	12/30 (40,0 %)
Pasos	60.000	240.000	0 nuevos; 120 jobs reanudados
Wall-clock	10.093,8 s	43.630,0 s	221,6 s de replay
Phase 2 throughput	6,04 steps/s	5,52 steps/s	n/a
Energía	290,507 Wh	1.191,626 Wh	n/a
Wh/1k pasos	4,8418	4,9651	n/a
CPU media	43,8 %	45,4 %	n/a
CPU saturada	0 %	0 %	n/a
GPU media	99,0 %	99,5 %	n/a
VRAM pico	8.331,1 MB	8.360,6 MB	n/a

Resultados por seed: 9/30, 9/30, 13/30 y 10/30. La unión oracle de las cuatro seeds contiene 14 tareas, pero no es una métrica válida porque usa ground truth para elegir.

La fusión raw resolvió exactamente las mismas 11 tareas que el control. La política v2 se materializó ejecutando `--resume` sobre los 120 parciales: cargó 120 jobs, dejó cero pendientes y Phase 2 duró 0,0 s. La submission final conserva esas 11 tareas y recupera `11852cab`, cuya solución correcta era top-2 de la seed 2 pero rango 3 tras la suma raw.

La política v2 se eligió después de inspeccionar el ground truth de este mismo demo. Aunque su ejecución sólo usa scores internos, **12/30 es un resultado post-hoc, no una estimación held-out**. Los intervalos Wilson 95 % son `[21,87 %, 54,49 %]` para 11/30 y `[24,59 %, 57,68 %]` para 12/30; con una ganancia y cero pérdidas, McNemar exacto bilateral da $p = 1,0$. La política debe pre-registrarse y probarse en tareas distintas antes de atribuirle mejora real.

Veredicto

- **Robustez:** aprobada. El gate 11/11 se mantiene y las trayectorias son diversas.
- **Mejora exploratoria post-hoc:** +1 tarea en los artefactos v2 mediante la política target-only v2, de 36,7 % a 40,0 %; no es estadísticamente concluyente.
- **Coste:** alto. Cuatro seeds multiplicaron wall-clock por 4,32 y energía total por 4,10. El throughput por paso cayó 8,6 % y Wh/1k pasos empeoró 2,5 %.
- **Cuello de botella:** no se trasladó a CPU; la saturación siguió en 0 % y la GPU permaneció al 99,5 %.
- **Alcance estadístico:** 30 tareas no permiten afirmar una mejora general de ARC-AGI. El siguiente experimento válido es un split mayor o una evaluación pre-registrada, no ajustar más la política sobre estas mismas 30 tareas.

Artefactos

- Control: `.profile/eje_b_demo30_control_summary.json`
- Candidato: `.profile/eje_b_demo30_candidate_summary.json`
- Metadata candidato: `.profile/eje_b_demo30_candidate_artifacts/run_metadata_training.json`
- Predicciones candidato: `.profile/eje_b_demo30_candidate_artifacts/predictions_training.npz`
- Parciales reanudables: `.profile/eje_b_demo30_candidate_artifacts/state/`
- Submission v2: `.profile/eje_b_demo30_candidate_v2_artifacts/submission_training.json`
- Predicciones v2: `.profile/eje_b_demo30_candidate_v2_artifacts/predictions_training.npz`
- Metadata v2: `.profile/eje_b_demo30_candidate_v2_artifacts/run_metadata_training.json`

Explicaion

Respuesta corta

Sí. En la ejecución original del demo:

$$30 \text{ tareas} \times 4 \text{ seeds} \times 2000 \text{ pasos} = 240\,000 \text{ pasos}$$

Cada combinación (`tarea`, `seed`) entrenó un modelo nuevo e independiente. La ejecución posterior de 221,6 s no reentrenó: cargó los 120 resultados guardados y recalculó la fusión v2.

Flujo Completo

1. `--demo 30` selecciona las primeras 30 tareas del split.
2. `--seeds 0,1,2,3` genera cuatro jobs por tarea en `parallel_train.py:403-421`.
3. Se crean 120 identificadores, por ejemplo:
 - `007bbfb7__seed_0`
 - `007bbfb7__seed_1`
 - ...
 - `1caeab9d__seed_3`
4. El scheduler ejecuta hasta seis simultáneamente, según VRAM y RAM. No se ejecutan los 120 a la vez.
5. Cada job nace en un proceso separado mediante `multiprocessing.Process` en `parallel_train.py:710-755`.
6. Dentro del proceso se fijan Python, NumPy, Torch y GPU al valor de la seed en `solve_task.py:57-76`.
7. Después se crean desde cero:
 - Un nuevo `Task`.
 - Un nuevo `ARCCompressor`.
 - Nuevos pesos aleatorios.
 - Un nuevo optimizador Adam.
 - Un nuevo logger y una EMA nueva.
8. El modelo ejecuta sus propios 2000 pasos en `solve_task.py:88-116`.

No se comparten pesos, gradientes, momentos de Adam, KL, curriculum ni logits entre seeds.

La seed 0 de una tarea tampoco transfiere nada a la seed 0 de otra tarea. “Seed 0” solo identifica una secuencia pseudoaleatoria; no es un modelo que vaya aprendiendo las 30 tareas.

Excepción: Phase 1

Phase 1 solo ejecuta dos pasos por tarea para medir memoria:

$$30 \times 2 = 60 \text{ pasos}$$

No repite esta medición para las cuatro seeds porque la forma del modelo es la misma. La independencia completa ocurre en Phase 2.

En la materialización v2 se leyeron los 120 parciales existentes. Por eso la metadata muestra `resumed_seed_jobs=120, trained_seed_jobs=0` y Phase 2 de 0 segundos.

Cambios En El Entrenamiento

Free-bits actúa por cada una de las 18 hojas KL:

$$L_{KL,t} = \sum_i \max(KL_i, \tau_t)$$

τ_t baja linealmente de 2 a 0. Si una hoja tiene $KL_i = 0,5$ y $\tau_t = 2$, aporta 2 a la pérdida, pero su gradiente compresivo es cero. Esto no la obliga a aprender; simplemente deja de empujarla hacia cero. Está implementado en `train.py`.

El curriculum calcula qué demostraciones fueron históricamente más difíciles. Les asigna más peso gradualmente:

$$w_i = n_{\text{train}} \text{softmax}(\beta_t d_i)$$

Los pesos tienen media 1. La dificultad usa una EMA detached de la pérdida por píxel. La entrada de test mantiene peso 1 y la salida oculta nunca participa en la pérdida. Véase [train.py](#) y [train.py](#).

Scoring Por Seed

Cada cuatro pasos se obtienen dos candidatos:

- Predicción actual.
- EMA de logits y máscaras.

Para cada candidato se calcula:

$$u = \text{mean}(\log \sum_c e^{l_c} - \max_c l_c) = - \text{mean}(\log p_{\text{color elegido}})$$

Después:

$$s = -10u - 10 \mathbf{1}_{t < 150} - 4 \mathbf{1}_{EMA}$$

Una solución segura recibe un score menos negativo. Si la misma cuadrícula aparece repetidamente, sus scores se acumulan mediante:

$$\text{logaddexp}(s_1, s_2) = \log(e^{s_1} + e^{s_2})$$

Esto premia frecuencia y confianza. Está en [solution_selection.py](#).

Fusión Entre Seeds

La fusión ocurre únicamente entre las cuatro ejecuciones de una misma tarea.

`attempt_1` usa consenso: los scores acumulados de una misma cuadrícula se combinan con `logaddexp`. Si varias seeds producen la misma respuesta, recibe más evidencia.

`attempt_2` busca diversidad. Primero normaliza dentro de cada seed:

$$\tilde{s}_{k,c} = s_{k,c} - \text{logsumexp}_{c'}(s_{k,c'})$$

Después toma para cada candidato su mejor soporte relativo:

$$D_c = \max_k \tilde{s}_{k,c}$$

Se elige la mejor solución distinta de `attempt_1`. La implementación está en `solution_selection.py:268-429`.

¿Es Válido?

Sí, respecto a ARC: el sistema puede ejecutar varios modelos y entregar finalmente solo dos respuestas. Esto es ensembling, no pass@4. El ground truth no participa en la fusión; se consulta después para calcular métricas en `parallel_train.py:1320-1368`.

Pero no es una probabilidad bayesiana rigurosa:

- El score es heurístico, no un logaritmo de probabilidad perfectamente calibrado.
- Las seeds usan los mismos ejemplos y arquitectura; son independientes en estado y RNG, pero no evidencia estadísticamente independiente.
- Los pasos consecutivos están muy correlacionados.
- `logaddexp` asume que los scores son comparables entre seeds.
- La normalización del segundo intento reduce este problema, pero no lo elimina.

Por tanto: **es una política de ranking válida y razonable, no una demostración probabilística.**

También es benchmark-valid, pero menos puro desde MDL: se buscan cuatro programas en vez de uno y se consume aproximadamente cuatro veces más cómputo.

Qué Mostraron Las Pruebas

- Seeds individuales: 9/30, 9/30, 13/30 y 10/30.
- Intersección de las cuatro: 7 tareas.
- Unión oracle: 14 tareas.
- Control: 11/30.
- Fusión raw: 11/30.
- Fusión v2: 12/30.

No se obtienen las 14 porque el sistema no conoce cuál seed acertó y solo puede entregar dos candidatos. La política v2 recuperó `11852cab`, pero fue diseñada tras inspeccionar este demo. Aunque durante la ejecución no usa ground truth, el 12/30 es post-hoc.

La validación científicamente correcta ahora es congelar esta política y probarla sin ajustes en un conjunto distinto. Solo entonces podremos afirmar que unificar scores mejora realmente la generalización.

Created 3 todos